

UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

ESTRUCTURAS DE DATOS II

MANUAL TECNICO PROYECTO 01

INTEGRANTES:

CRISTHIAN ESTUARDO DE LEÓN PÉREZ

CARNET:

1540225

QUETZALTENANGO, SEPTIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S.J

DIAGRAMA DE FLUJO

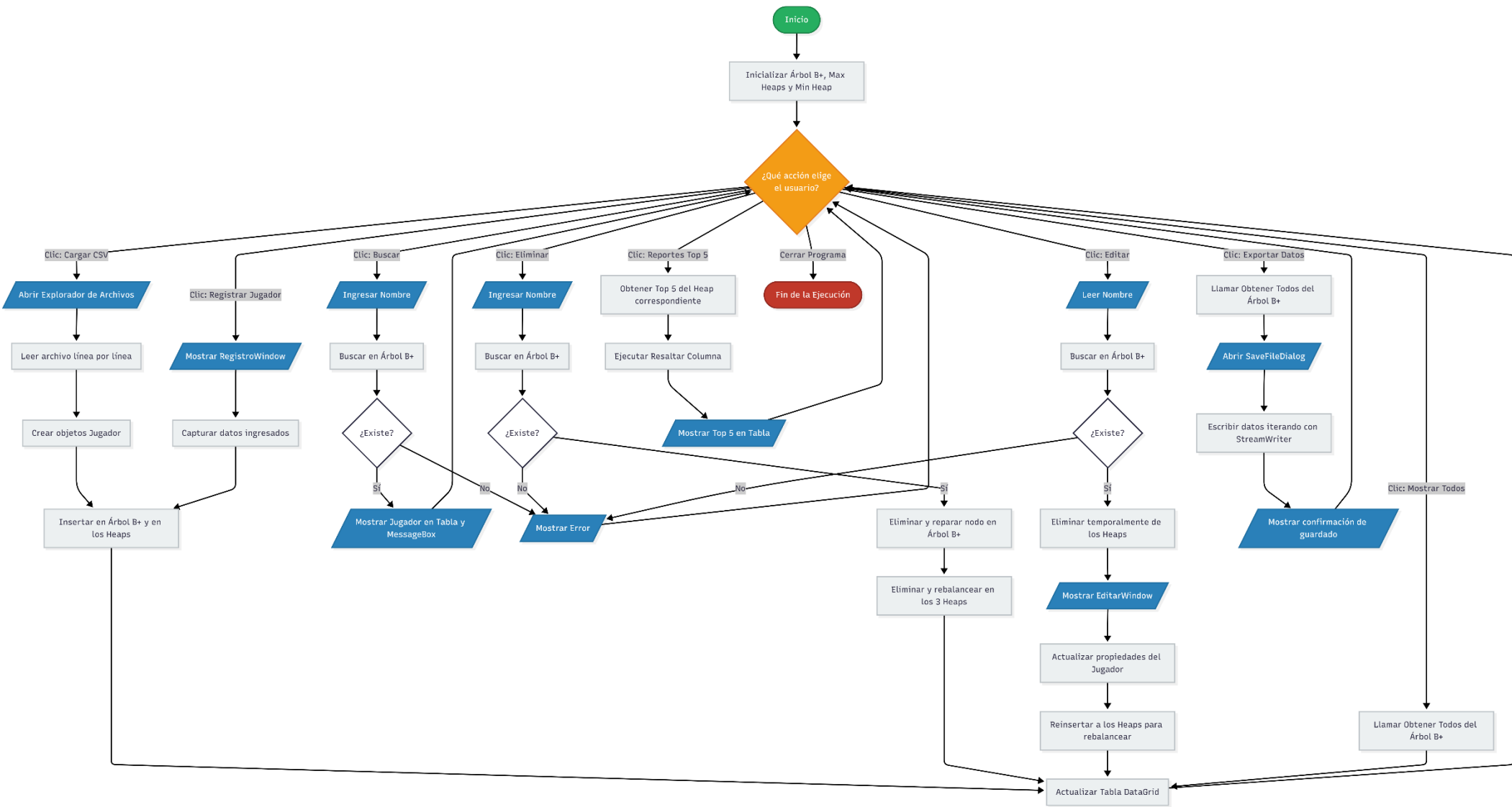
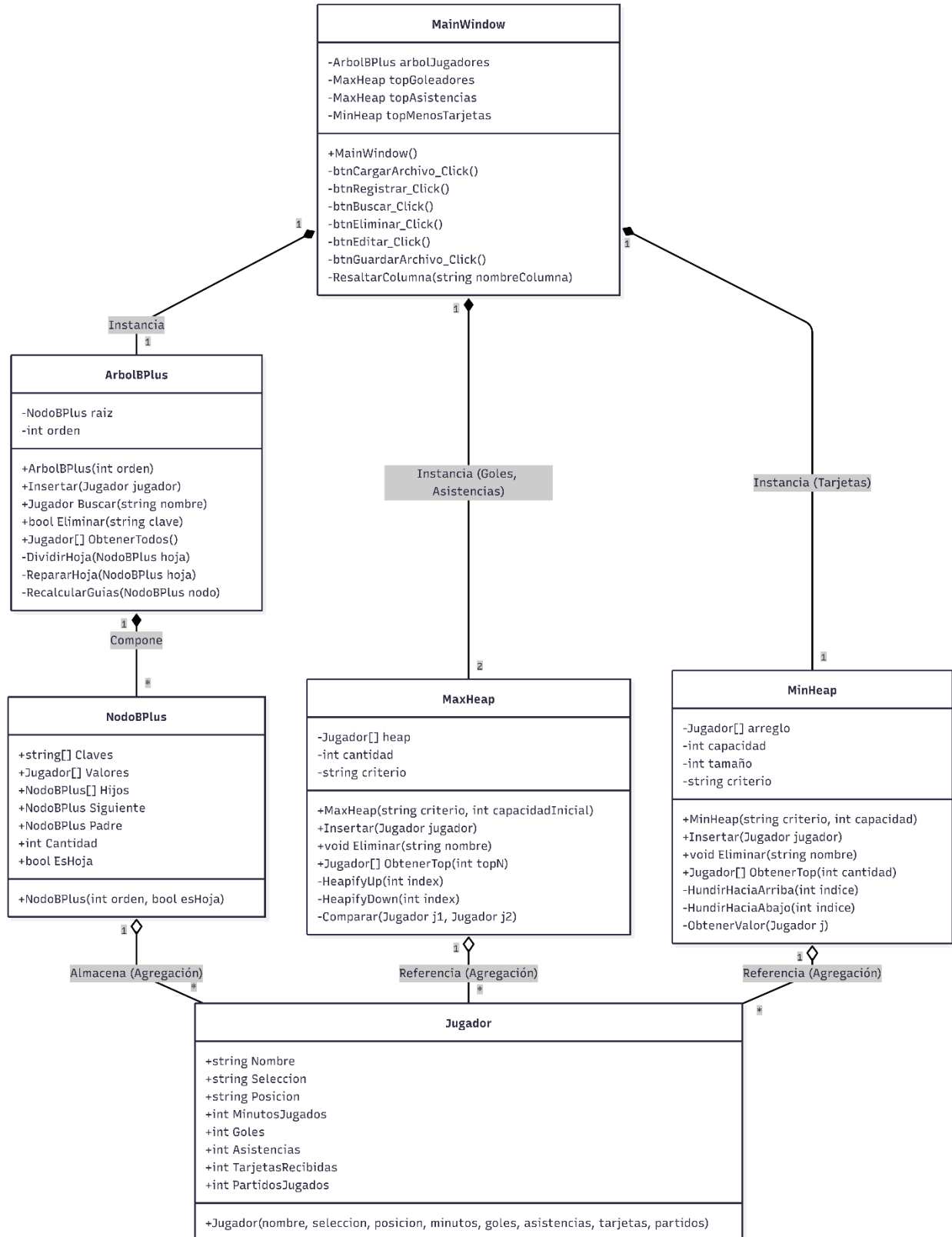


DIAGRAMA DE CLASE



ESTRUCTURA DEL PROGRAMA

El presente proyecto para el curso de Estructura de Datos II se desarrollo un programa sobre el Mundial, el mismo se desarrolló en el lenguaje C#. Para el almacenamiento, procesamiento y ordenamiento de los datos, se implementaron desde cero tres estructuras jerárquicas: un Árbol B+, Max Heap y Min Heap.

ÁRBOL B+

Se utilizó el Árbol B+ ya que funciona como el motor principal de almacenamiento donde los objetos Jugador se organizan alfabéticamente utilizando su atributo Nombre como clave principal. La estructura separa lógicamente la navegación de los datos reales, los nodos internos contienen únicamente arreglos de texto (string, Claves) que actúan como "letreros" para dirigir la búsqueda rápidamente, mientras que los nodos hoja almacenan los arreglos con los objetos completos (Jugador, Valores). Esta arquitectura, sumada al puntero Siguiente que enlaza todas las hojas formando una lista continua, es lo que permite que el programa pueda encontrar a un jugador específico desde la barra de búsqueda en tiempo $O(\log n)$, y al mismo tiempo, utilizar el método ObtenerTodos() para recorrer secuencialmente las hojas y llenar la tabla DataGridView o exportar el archivo CSV de forma instantánea $O(n)$, ajustando y desplazando los índices de los arreglos automáticamente cada vez que se registra o elimina un futbolista.

MAX HEAP

En el programa, el Max Heap funciona como un índice secundario especializado que gestiona de forma automática los reportes de "Top 5 Goleadores" y "Top 5 Asistencias". En lugar de usar una estructura de nodos con punteros, se implementa puramente de forma matemática sobre un arreglo estático (Jugador), donde la posición de cada jugador determina su jerarquía: el elemento en el índice i tiene a sus hijos en las posiciones $2i + 1$ y $2i + 2$, y a su padre en $(i - 1)/2$. Cada vez que se registra, edita o elimina un futbolista en la interfaz, los métodos internos (HeapifyUp y HeapifyDown) comparan la estadística específica del criterio seleccionado y realizan intercambios directamente en el arreglo para garantizar que el jugador con el número más alto siempre suba hasta la raíz (índice 0). Esto permite que, al presionar el botón del reporte, el sistema pueda extraer a los cinco mejores jugadores casi al instante, con un acceso inicial de $O(n)$ y extracciones de $O(\log n)$, evitando por completo el tener que ordenar toda la base de datos general.

MIN HEAP

Por ultimo, el Min Heap comparte la misma arquitectura matemática sobre un arreglo estático que el Max Heap, pero opera con la lógica de ordenamiento invertida para gestionar exclusivamente el reporte de "Top 5 Menos Tarjetas". Utilizando la misma navegación por índices (hijos en $2i + 1$ y $2i + 2$, padre en $(i - 1)/2$), sus métodos internos de rebalanceo evalúan la estadística de tarjetas recibidas de cada futbolista. La diferencia fundamental radica en que las comparaciones obligan al jugador con el número menor de tarjetas subir hacia la raíz del arreglo (índice 0). De esta forma, el sistema mantiene un índice secundario especializado que aísla a los

jugadores más disciplinados, permitiendo extraer a los cinco mejores de manera óptima y consecutiva con un costo de $O(\log n)$ por extracción, evitando por completo la necesidad de recorrer la base de datos principal del Árbol B+ en busca de los valores mínimos.